

NẠP CHỒNG TOÁN TỬ

1

1

VÍ DỤ

- ❑ Xây dựng lớp phân số có thể thực hiện các phép toán số học : cộng, trừ, nhân chia, ...
- ❑ Có hai cách xây dựng các phép toán
 - Dùng phương thức :
`Faction theSum,firstFraction, secondFraction;`
`the Sum = firstFraction.add(secondFraction);`
 - Dùng toán tử
`Faction theSum,firstFraction, secondFraction;`
`the Sum = firstFraction+secondFraction;`

2

2

SỬ DỤNG TỪ KHÓA operator

- ❑ Các toán tử là các phương thức tĩnh. Giá trị trả về thể hiện kết quả của một toán tử và các tham số là toán hạng.
- ❑ Tạo một toán tử cho một lớp tức là thực hiện việc nạp chồng.
- ❑ Toán tử nạp chồng được khai báo sử dụng từ khóa operator

■ Ví dụ

```
class Fraction {
    int x, y;
    public Fraction (int x, int y) {this.x = x; this.y = y; }
    public static Fraction operator + (Fraction a, Fraction b) {
        return new Fraction(a.x * b.y + b.x * a.y, a.y * b.y);
    }
}
```

■ Sử dụng

```
Fraction a = new Fraction(1, 2);
Fraction b = new Fraction(3, 4);
Fraction c = a + b; // c.x == 10, c.y == 8
```

3

3

MỘT SỐ LUẬT

- ❑ Định nghĩa những toán tử trong kiểu dữ liệu giá trị, kiểu do ngôn ngữ xây dựng sẵn
- ❑ Cung cấp những phương thức nạp chồng toán tử chỉ bên trong của lớp nơi mà những phương thức được định nghĩa.
- ❑ Sử dụng tên và những ký hiệu qui ước mô tả trong Common Language Specification (CLS)
- ❑ Phải cung cấp các phương thức thay thế cho toán tử nạp chồng

4

4

MỘT SỐ LUẬT		
Biểu tượng	Tên phương thức thay thế	Tên toán tử
+	Add	Toán tử cộng
-	Subtract	Toán tử trừ
*	Multiply	Toán tử nhân
/	Divide	Toán tử chia
%	Mod	Toán tử chia lấy dư
^	Xor	Toán tử or loại trừ
&	BitwiseAnd	Toán tử and nhị phân
	BitwiseOr	Toán tử or nhị phân
&&	And	Toán tử and logic
	Or	Toán tử or logic
=	Assign	Toán tử gán
<<	LeftShift	Toán tử dịch trái
>>	RightShift	Toán tử dịch phải

5

MỘT SỐ LUẬT		
Biểu tượng	Tên phương thức thay thế	Tên toán tử
==	Equals	Toán tử so sánh bằng
>	Compare	Toán tử so sánh lớn hơn
<	Compare	Toán tử so sánh nhỏ hơn
!=	Compare	Toán tử so sánh khác
>=	Compare	Toán tử so sánh lớn hơn hoặc bằng
<=	Compare	Toán tử so sánh nhỏ hơn hoặc bằng
*=	Multiply	Toán tử nhân và gán lại
-=	Subtract	Toán tử trừ và gán lại
^=	Xor	Toán tử or loại trừ và gán lại
<<=	LeftShift	Toán tử dịch trái và gán lại
%=	Mod	Toán tử chia dư rồi gán lại

6

MỘT SỐ LUẬT

Biểu tượng	Tên phương thức thay thế	Tên toán tử
+=	Add	Toán tử cộng rồi gán lại
&=	BitwiseAnd	Toán tử and rồi gán lại
=	BitwiseOr	Toán tử or rồi gán lại
/=	Divide	Toán tử chia rồi gán lại
--	Decrement	Toán tử giảm
++	Increment	Toán tử tăng
-	Negate	Toán tử phủ định một ngôi
+	Plus	Toán tử cộng một ngôi
~	OnesComplement	Toán tử bù

7

7

TOÁN TỬ SO SÁNH BẰNG

- ❑ Nếu nạp chồng toán tử so sánh bằng (==), nên phủ quyết phương thức ảo Equals của lớp object và chuyển lại cho toán tử so sánh bằng thực hiện.
- ❑ Ví dụ


```
public override bool Equals(object o)
{
    if ( !(o is Fraction) ) return false;
    return this == (Fraction) o;
}
```
- ❑ Nạp chồng toán tử có tính đối xứng, tức là nếu toán tử so sánh bằng được nạp chồng thì cũng phải nạp chồng toán tử không bằng (!=).
- ❑ Nếu toán tử (==) và (!=) mà được nạp chồng và phương thức Equals không được nạp chồng thì chương trình sẽ cảnh báo.

8

8

VÍ DỤ

```
using System;
public class Fraction
{
    public Fraction(int numerator,int denominator)
    {
        Console.WriteLine("In Fraction Constructor( int, int) ");
        this.numerator = numerator;
        this.denominator = denominator;
    }
    public Fraction(int wholeNumber)
    {
        Console.WriLine("In Fraction Constructor( int )");
        numerator = wholeNumber;
        denominator = 1;
    }
    public static implicit operator Fraction( int theInt )
    {
        Console.WriteLine(" In implicit conversion to Fraction");
        return new Fraction( theInt );
    }
}
```

9

9

VÍ DỤ

```
public static explicit operator int( Fraction theFraction )
{
    Console.WriteLine("In explicit conversion to int");
    return theFraction.numerator / theFraction.denominator;
}
public static bool operator == ( Fraction lhs, Fraction rhs)
{
    Console.WriteLine("In operator ==");
    if ( lhs.numerator == rhs.numerator &&
        lhs.denominator == rhs.denominator ) return true;
    // thực hiện khi hai phân số không bằng nhau
    return false;
}
public static bool operator != ( Fraction lhs, Fraction rhs)
{
    Console.WriteLine("In operator !=");
    return !( lhs == rhs );
}
public override bool Equals( object o )
{
    Console.WriteLine("In method Equals");
    if ( !(o is Fraction ) ) return false;
    return this == ( Fraction ) o;
}
```

10

10

VÍ DỤ

```
public static Fraction operator+( Fraction lhs, Fraction rhs )
{
    Console.WriteLine("In operator +");
    if (lhs.denominator == rhs.denominator )
        return new Fraction( lhs.numerator + rhs.numerator,
                               lhs.denominator );
    //thực hiện khi hai mẫu số không bằng nhau
    int firstProduct = lhs.numerator * rhs.denominator;
    int secondProduct = rhs.numerator * lhs.denominator;
    return new Fraction( firstProduct + secondProduct,
                          lhs.denominator * rhs.denominator);
}
public override string ToString()
{
    string s = numerator.ToString() + "/" + denominator.ToString();
    return s;
}
//biến thành viên lưu tử số và mẫu số
private int numerator;
private int denominator;
}
```

11

11

VÍ DỤ

```
public class Tester
{
    static void Main()
    {
        Fraction f1 = new Fraction( 3, 4);
        Console.WriteLine("f1:{0}",f1.ToString());
        Fraction f2 = new Fraction( 2, 4);
        Console.WriteLine("f2:{0}",f2.ToString());
        Fraction f3 = f1 + f2;
        Console.WriteLine("f1 + f2 = f3:{0}",f3.ToString());
        Fraction f4 = f3 + 5;
        Console.WriteLine("f4 = f3 + 5:{0}",f4.ToString());
        Fraction f5 = new Fraction( 2, 4);
        if( f5 == f2 )
        {
            Console.WriteLine("f5:{0}==f2:{1}",
                              f5.ToString(), f2.ToString());
        }
    }
}
```

12

12